

MYNDRA-LWM: A Practitioner Taxonomy of Security Risks in Large World Model Systems

Spyros Argyraeos

Senior Cybersecurity Engineer, AI Security Lead

MYNDRA Framework

spyrosarg.github.io/myndra-lwm

2026

Abstract

As autonomous AI systems increasingly rely on internal world models to perceive, reason, and act in physical environments, a critical security gap has emerged: no practitioner framework exists for systematically identifying and mitigating attacks targeting the cognitive architecture these systems depend on. Existing frameworks, including OWASP LLM Top 10 and MITRE ATLAS, address language model applications and adversarial machine learning respectively, but do not cover the distinct failure modes of systems that build and act on internal causal representations of reality. I introduce MYNDRA-LWM, an open practitioner taxonomy of security risks in Large World Model systems. Organized around five cognitive architecture components, perception, world state, prediction, planning, and action, the taxonomy defines ten attack categories, each with formal definition, attack vector, detection guidance, and mitigation strategies. MYNDRA-LWM is architecture-agnostic and applies across autonomous vehicles, robotic systems, spacecraft, and AI agents with persistent world state.

1 Introduction

The dominant paradigm in deployed artificial intelligence is shifting. For the past decade, security practitioners have focused on systems that process language, models that receive text, generate text, and whose failure modes are largely linguistic in character. This focus produced valuable frameworks: the OWASP Top 10 for Large Language Models, MITRE ATLAS, and a growing body of adversarial machine learning research. These frameworks address real threats and have materially improved the security posture of language model deployments worldwide.

A different class of system is now entering production. Autonomous vehicles, robotic manipulators, spacecraft with onboard decision-making capability, and AI agents with persistent environmental memory share a common architectural property: they do not merely process descriptions of the world. They construct internal models of it. These systems perceive raw sensory streams, compress observations into compact latent representations, learn the causal dynamics of their environments, plan sequences of actions by simulating futures internally, and execute those plans in physical reality. The security implications of this architecture are fundamentally different from those of language model systems, and remain almost entirely unaddressed.

The attack surface of a world model system is not its inputs in the linguistic sense. It is the fidelity of the system’s internal representation of reality. An adversary who corrupts what a world model believes about its environment, without touching a single line of code or model parameter, can redirect the system’s behavior as effectively as any classical exploit. The perception encoder, the latent state representation, the transition dynamics model, the planning objective, and the reward signal each constitute a distinct attack surface with distinct failure modes. None

of these components appear in existing security frameworks organized around language model vulnerabilities.

This paper introduces MYNDRA-LWM, an open practitioner taxonomy of security risks in Large World Model systems. The taxonomy organizes ten attack categories around the five cognitive components that define the world model architecture: perception, world state, prediction, planning, and action. For each category, I provide a formal definition of the attack, a concrete attack vector, detection guidance, and mitigation strategies. The framework is architecture-agnostic, applicable to any system whose behavior is governed by an internal causal model of its environment, and is structured to complement rather than duplicate existing frameworks covering language model and agentic AI systems.

The remainder of this paper is organized as follows. Section 2 describes the world model cognitive architecture that forms the taxonomic basis. Section 3 presents the MYNDRA-LWM Top 10 taxonomy. Section 4 discusses usage guidelines for threat modeling, red teaming, and secure design. Section 5 defines the scope and limitations of the framework. Section 6 concludes.

2 Background: The World Model Cognitive Architecture

The term world model describes an AI system that maintains an internal representation of its environment and uses that representation to reason about consequences before acting. The concept has roots in cognitive science and control theory, but has gained significant traction in machine learning research as a principled path toward autonomous systems capable of long-horizon planning in complex physical environments.

Unlike language models, which learn statistical relationships between tokens in text, world model systems learn from direct interaction with environments, processing raw sensory observations and building compressed representations that capture the causal structure of how that environment behaves. The distinction is not merely architectural. It reflects a fundamentally different relationship between the system and reality: language models learn how humans describe the world, while world model systems learn how the world actually works.

Regardless of specific implementation, deployed world model systems share a common cognitive structure. This structure is the basis on which MYNDRA-LWM organizes its taxonomy.

Perception. The perception component encodes raw environmental observations into compact latent representations. Inputs may include visual frames, sensor readings, audio streams, control signals, or telemetry. The encoder compresses this high-dimensional input into a lower-dimensional representation that captures the features relevant to the system’s task. All downstream reasoning depends entirely on the quality and accuracy of this encoding.

World state. The world state component maintains the system’s current belief about its environment. This is the latent representation of what the system understands to be true about the world at the present moment, constructed from the encoded observation and integrated with prior beliefs accumulated over time. The world state is the substrate on which all prediction and planning operate.

Prediction. The prediction component, often called the transition model or dynamics model, learns how world states evolve in response to actions. Given the current world state and a candidate action, the predictor estimates the resulting next state. This component encodes the system’s causal understanding of its environment, the learned model of what causes what. Planning depends entirely on the fidelity of these predictions.

Planning. The planning component uses the predictive model to evaluate candidate action sequences by simulating their outcomes internally, without acting in the real environment. By

rolling out sequences of predicted states, the planner identifies actions that are expected to achieve the system’s objectives. This simulation capability is what distinguishes world model systems from reactive agents: they reason about consequences before committing to action.

Action and reward. The action component executes the selected action in the real environment and receives the resulting observation, closing the perception-action loop. The reward signal, whether specified explicitly or learned from preference data, governs which outcomes the system optimizes toward. It is the ultimate arbiter of system behavior across all planning horizons.

These five components form a closed loop. An attack on any single component propagates through the entire system. A corrupted encoder produces a false world state. A false world state produces incorrect predictions. Incorrect predictions produce suboptimal or adversarial plans. Adversarial plans produce harmful actions. This cascade is the fundamental security property that makes world model systems a distinct and coherent attack surface, one that existing frameworks, designed around stateless input-output transformations, do not address.

3 The MYNDRA-LWM Top 10

Table 1 summarizes the ten attack categories, their target cognitive component, and severity rating.

ID	Name	Component	Severity
WM-01	Perception Attack	Perception	Critical
WM-02	Latent Poisoning	World state	Critical
WM-03	Transition Attack	Prediction	High
WM-04	Planning Attack	Planning	Critical
WM-05	Reward Hijack	Action	Critical
WM-06	Sim-to-Real Gap Exploitation	World state	High
WM-07	Belief Corruption	Prediction	High
WM-08	Surprise Bypass	Perception	Critical
WM-09	Model Extraction	World state	Medium
WM-10	Collapse Attack	Planning	Critical

Table 1: MYNDRA-LWM Top 10 attack categories

3.1 WM-01: Perception Attack

Definition. A Perception Attack corrupts the encoding boundary between raw environmental observations and the world model’s internal representation. By introducing crafted perturbations into the input stream, the attacker causes the encoder to produce systematically incorrect latent representations while the inputs appear normal to human reviewers. Because all downstream reasoning depends on the perception layer, corruption at this boundary propagates silently through the entire cognitive stack.

Attack vector. The attacker probes the relationship between raw inputs and encoded representations, estimates the gradient of the encoding loss with respect to input perturbations, and generates perturbations that produce maximally corrupted encodings while remaining below human detection thresholds. In physical deployments, perturbations are introduced through adversarial patches, sensor signal manipulation, or modified environmental features. The corrupted encoding cascades into the world state, prediction, and planning components without triggering architectural defenses designed for higher layers.

Detection. Deploy multiple independent encoders and monitor divergence between their outputs for the same observation. For multi-sensor systems, verify consistency between independent

sensory channels. Track the statistical distribution of encoded representations over time and alert on distributional drift without corresponding environmental change.

Mitigation. Train encoders with certified adversarial robustness guarantees. Cross-validate between independent sensor modalities. For high-stakes deployments, authenticate sensor outputs at the hardware level through cryptographic signing, preventing controlled perturbation of the input stream.

3.2 WM-02: Latent Poisoning

Definition. Latent Poisoning corrupts the geometry or content of the world model’s learned latent representation space, the compressed mathematical encoding of world state that the system develops through training. Unlike perception attacks that target the raw input boundary, this attack operates on the representation itself, inducing systematic errors that propagate through all reasoning while appearing as valid world states to the system’s own consistency checks.

Attack vector. At training time, the attacker injects crafted data that shifts latent geometry toward adversarial regions. At inference time, inputs are crafted to map to adversarial positions in the latent manifold despite appearing benign in raw input space. Backdoor triggers embedded during training activate adversarial latent regions when specific patterns are present at deployment.

Detection. Monitor the statistical properties and geometry of latent representations during operation. Maintain lightweight probe classifiers that predict physical quantities from latent embeddings and alert on degraded probe accuracy. Implement cryptographic verification of training data origins.

Mitigation. Cryptographically sign training data at collection and verify signatures throughout the data pipeline. Apply regularization during training that enforces structural properties incompatible with poisoning goals. Use differentially private training procedures that limit the influence any individual training example can have on the learned representation.

3.3 WM-03: Transition Attack

Definition. A Transition Attack corrupts the world model’s learned causal dynamics, breaking its ability to predict how states evolve in response to actions. The system continues operating by its own metrics while pursuing trajectories shaped by the attacker, because planning depends entirely on the accuracy of transition predictions and has no independent ground truth against which to verify them.

Attack vector. Through repeated behavioral probing, the attacker characterizes the transition function’s learned dynamics and identifies regions of the state-action space where learned dynamics diverge from ground truth. The attacker then designs sequences that exploit these gaps to steer the system toward target states while keeping individual steps within the learned distribution. The system, unaware it is operating at the boundary of its training distribution, extrapolates its learned dynamics incorrectly, steering itself toward the attacker’s target state through its own planning loop.

Detection. Track the actual prediction error of the transition model in real time, the divergence between predicted next states and observed next states. Verify that transition predictions satisfy known physical constraints and invariants. Maintain multiple independent transition models and alert on high divergence between ensemble predictions.

Mitigation. Train transition models that explicitly represent uncertainty about their own predictions, triggering conservative fallback behavior in high-uncertainty states. For physical systems, embed hard constraints from known physics into the transition model architecture as

structural properties rather than learned behaviors. Periodically test the transition model against held-out ground-truth trajectories.

3.4 WM-04: Planning Attack

Definition. A Planning Attack corrupts the planning mechanism to redirect long-horizon behavior while leaving the underlying world model intact. By manipulating goal representations, objective functions, or the planning algorithm itself, the attacker redirects the system’s optimization toward chosen outcomes. The attack is difficult to detect because perception, state estimation, and transition prediction all function correctly.

Attack vector. The attacker identifies how the system’s planning goals are represented and introduces corrupted goal representations through input manipulation, configuration poisoning, or reward signal interception. A corrupted goal encoding, when optimized toward, produces the attacker’s desired behavioral trajectory. Planning horizon manipulation forces myopic decision-making by corrupting the temporal discount structure.

Detection. Implement cryptographic verification of goal representations before they enter the planning loop. Track the statistical distribution of planned action sequences over time and alert on systematic drift toward unusual trajectories. For high-stakes deployments, use an independent system to verify that the stated goal aligns with the system’s behavioral trajectory before execution.

Mitigation. Cryptographically sign goal representations at the point of specification. Run multiple independent planning processes and require consensus on selected action sequences before execution. For systems where goals are specified externally, require human review and approval before passing goals to the planning system.

3.5 WM-05: Reward Hijack

Definition. A Reward Hijack corrupts the reward signal that governs which outcomes the world model system optimizes toward, redirecting the entire optimization process toward attacker-chosen objectives. The attack is classified under the action component because the reward signal is the ultimate governor of which actions the system selects, corrupting it directly corrupts the system’s decision at the moment of execution. The system continues to function correctly by its own metrics, optimizing effectively toward the wrong objective.

Attack vector. The attacker maps the reward signal pipeline and identifies points where adversarial influence is possible, including training data injection, preference label manipulation, or direct reward model parameter access. For learned rewards, crafted training examples shift the reward model toward the target objective. For specified rewards, the signal is intercepted and modified. The system converges on the attacker’s target behavioral equilibrium through its own optimization process.

Detection. Periodically evaluate the reward model against held-out ground-truth preferences and alert on significant divergence from calibration baselines. Track whether behavioral outputs are consistent with the stated reward specification across diverse scenarios. Implement cryptographic verification and auditing for all preference data used to train reward models.

Mitigation. Aggregate reward signals from multiple independent sources using robust aggregation methods. Implement hard constraints on the reward model’s output distribution that are architectural rather than learned. Before deployment, systematically test the reward model against known reward poisoning strategies.

3.6 WM-06: Sim-to-Real Gap Exploitation

Definition. Every world model system is trained on a finite distribution of experiences that cannot perfectly represent every possible deployment condition. Sim-to-Real Gap Exploitation deliberately targets the boundary of this distribution, constructing conditions that fall outside what the model has learned, causing it to apply incorrect learned dynamics to genuinely novel situations. The attack requires no access to model parameters, only knowledge of the training distribution’s boundaries.

Attack vector. Through behavioral probing and domain knowledge, the attacker estimates the boundaries of the training distribution and identifies configurations that are physically realizable in the deployment environment but absent from training. The attacker engineers these out-of-distribution conditions in the physical or operational environment. The world model, lacking calibrated uncertainty in this region, applies learned dynamics inappropriately and fails in ways that are difficult to anticipate from within the training distribution.

Detection. Deploy a lightweight out-of-distribution detector that monitors whether current observations fall within the training distribution. Use Bayesian or ensemble methods that explicitly represent epistemic uncertainty and flag high-uncertainty states. Continuously characterize the statistical properties of the deployment environment and alert on significant distributional shift.

Mitigation. Train across a deliberately widened distribution of environment conditions through domain randomization. Design systems to fall back to conservative pre-specified behaviors when epistemic uncertainty exceeds thresholds. Implement constrained online learning that allows the world model to update its representations during deployment within safety-verified bounds.

3.7 WM-07: Belief Corruption

Definition. World model systems maintain persistent beliefs accumulated across an extended operational lifetime. Belief Corruption exploits this persistence through sustained campaigns of calibrated adversarial observations that incrementally shift the accumulated belief state toward the attacker’s target. Each individual observation may appear normal, but their cumulative effect systematically corrupts the model’s integrated world state in ways that evade point-in-time anomaly detectors.

Attack vector. The attacker characterizes how the system integrates new observations into its persistent belief state and plans a temporal sequence of observations that individually fall within normal ranges but cumulatively drive the belief state toward the target. The campaign is paced to stay within the belief update bandwidth that avoids detection, operating across days, weeks, or multiple operational sessions.

Detection. Maintain cryptographically signed snapshots of the system’s belief state at regular intervals and compare current beliefs against historical baselines to detect gradual corruption. Periodically run standardized behavioral tests against known ground-truth scenarios and alert on degraded performance. Cross-validate between independently maintained belief components.

Mitigation. Maintain cryptographically sealed belief state snapshots with rollback capability. Constrain the maximum rate at which new observations can shift persistent beliefs, imposing temporal robustness that requires longer attack campaigns and increases detection probability. Train the belief update mechanism with adversarial sequential examples specifically designed to induce gradual corruption.

3.8 WM-08: Surprise Bypass

Definition. Every world model system measures the divergence between what it predicted and what it observed, a surprise signal that serves as the primary safety sensor. A Surprise Bypass

attack manipulates this signal, making genuinely dangerous or physically implausible events register as expected. The safety layer fails silently: the system perceives the dangerous state, rates it as normal, and proceeds without defensive response.

Attack vector. The attacker probes the world model to map how observations produce surprise scores, estimates the gradient of the surprise signal with respect to input perturbations, and deploys a temporal sequence of observations that incrementally shift the system’s baseline expectations. Each step is small enough to avoid triggering the surprise threshold individually, but cumulatively the model’s expectations drift toward the target anomaly. At the point of exploitation, the target dangerous observation is generated with a perturbation that minimizes its surprise score against the corrupted baseline.

Detection. Track the statistical distribution of surprise scores per observation category over rolling time windows and alert on systematic drift toward lower values without documented environmental changes. Periodically inject known-implausible synthetic observations whose correct surprise score is established at training time. Maintain multiple independent predictors and alert when an observation generates low surprise in the primary model but high surprise across ensemble members.

Mitigation. Deploy multiple independent world models and require consensus before an observation is rated non-surprising. Maintain a cryptographically sealed baseline of surprise score distributions per observation category, updated only under audit conditions. During training, include explicit Surprise Bypass attack examples to train calibrated surprise scores under adversarial conditions.

3.9 WM-09: Model Extraction

Definition. The internal structure of a world model, its learned causal dynamics, latent geometry, and physical priors, represents significant intellectual and strategic value. A Model Extraction attack systematically recovers this internal structure through carefully crafted query sequences without requiring direct access to model parameters. The recovered surrogate model enables more effective attacks against the original, competitive intelligence gathering, and circumvention of access controls on proprietary capabilities.

Attack vector. The attacker designs a systematic query strategy that maximally reveals internal model structure, including latent space geometry, transition dynamics, and uncertainty estimates. Collected input-output pairs are used to train a surrogate model that approximates the original’s behavior. The surrogate enables white-box adversarial attacks against a target that only exposes a black-box interface.

Detection. Monitor query patterns for systematic characteristics suggesting extraction intent rather than operational use, including structured coverage of the input space and progressive refinement between queries. Log all queries for forensic analysis. Implement per-user query rate limits that bound the information extractable within a given time window.

Mitigation. Apply differential privacy to model outputs, providing formal guarantees on the amount of information extractable through any query strategy. Implement output perturbation schemes that degrade extraction quality while preserving task-relevant accuracy. Embed detectable watermarks in model behavior that allow attribution of extracted surrogates back to the source model.

3.10 WM-10: Collapse Attack

Definition. Representation collapse is the catastrophic failure mode where the world model’s latent space loses its meaningful geometric structure, all observations map to identical or near-

identical representations, and the system can no longer distinguish between different world states. A Collapse Attack deliberately induces this failure through adversarial inputs designed to defeat the regularization mechanisms that maintain latent space structure. Although the attack vector operates at the representation layer, it is classified under planning because planning is the first cognitive function to fail completely and irrecoverably when latent geometry collapses.

Attack vector. The attacker identifies the regularization methods the system uses to prevent representation collapse, including Gaussian regularizers, contrastive losses, or variance constraints. Inputs are crafted that, when processed through the encoder, maximally violate the regularizer’s assumptions, driving the encoder toward producing near-identical representations for diverse inputs. The attack is calibrated to find the minimum intensity required to induce collapse while remaining below detection probability on surrogate models before targeting the live system.

Detection. Continuously track the variance and geometric diversity of latent representations during operation. Rapid decline in representational diversity is the earliest indicator of collapse induction. Monitor downstream task performance metrics that require representational diversity. Implement a dedicated real-time collapse early warning system that triggers alerts before collapse becomes operationally significant.

Mitigation. Use training objectives that combine multiple complementary regularization terms with different failure modes, making simultaneous defeat significantly harder. Implement structural mechanisms in the encoder architecture that resist collapse as architectural properties rather than learned behaviors. Continuously monitor representational health and implement automatic fallback to a last-known-good model checkpoint when collapse indicators exceed thresholds.

4 Usage Guidelines

MYNDRA-LWM is designed to be applied across three primary security workflows: threat modeling, red teaming, and secure design. The framework is composable with existing security standards and does not require replacement of existing processes.

4.1 Threat modeling

The five-component cognitive architecture provides a structured basis for threat modeling world model systems. For each component present in the target system, the corresponding WM entries identify the relevant attack categories and their associated risk levels. A system that exposes a perception encoder, a learned transition model, and a planning module based on goal-image conditioning is exposed to WM-01, WM-03, WM-04, WM-08, and potentially WM-10. A system that additionally maintains persistent belief state across operational sessions adds WM-02, WM-06, and WM-07 to the threat surface. Threat modelers should walk the architecture diagram in Section 2 systematically, mapping each deployed component to its corresponding WM entries and documenting the resulting attack surface before mitigation analysis begins.

4.2 Red teaming

Each WM entry’s attack vector section provides a concrete methodology for testing the corresponding vulnerability in a deployed system. Red team exercises using MYNDRA-LWM should be structured as follows. First, identify which cognitive components are accessible from the attacker’s assumed position, whether external API access, physical environment access, or supply chain access to training data or model artifacts. Second, select the corresponding WM entries and follow each attack vector methodology in sequence, from reconnaissance through exploitation. Third, use the detection section of each entry to identify what monitoring signals should have fired during the test but did not, as detection gaps are often as operationally significant as the

vulnerabilities themselves. Results should be documented per WM entry with evidence of success or failure and the specific conditions under which each attack was or was not feasible.

4.3 Secure design

Applied at design time, MYNDRA-LWM functions as a security requirements checklist organized by cognitive component. For each component included in a planned architecture, the corresponding mitigation sections define the security properties that component should satisfy before deployment. Several mitigations in the taxonomy are significantly cheaper to implement as design decisions than as retrofits to deployed systems. Ensemble architectures that provide cross-validation between independent components, cryptographic authentication of goal representations and sensor outputs, and uncertainty-aware transition models that trigger conservative fallback behavior are all substantially easier to incorporate during initial design than after the system is in production. Secure design reviews should verify that each planned cognitive component has addressed the relevant mitigations before the system proceeds to deployment testing.

4.4 Relationship to existing frameworks

MYNDRA-LWM is designed to complement rather than replace existing security frameworks. For systems that combine world model components with language model interfaces, OWASP LLM Top 10 applies to the language interface layer while MYNDRA-LWM applies to the world model cognitive stack beneath it. For organizations using MITRE ATLAS for adversarial machine learning coverage, MYNDRA-LWM provides the world-model-specific layer that ATLAS does not address. MYNDRA-LWM entries can be mapped to MITRE ATT&CK tactics at the practitioner’s discretion, though no formal mapping is provided in this version of the framework.

5 Scope and Limitations

5.1 Scope

MYNDRA-LWM covers decision-coupled world model systems, defined as architectures where the world model’s internal representations and predictions directly govern autonomous action in the real world. This definition includes robotic manipulation systems, autonomous ground and aerial vehicles, spacecraft with onboard autonomous decision-making capability, and AI agents that maintain persistent belief state across operational sessions and use that state to select actions with real-world consequences.

The framework applies regardless of the specific implementation architecture. Systems based on recurrent state space models, joint embedding predictive architectures, diffusion-based world models, or hybrid neural-symbolic approaches all share the five-component cognitive structure described in Section 2 and are within scope. The organizing principle is functional, not architectural: if the system builds an internal model of its environment and uses that model to plan actions, MYNDRA-LWM applies.

5.2 Out of scope

Generative video models trained to produce visually coherent output for human consumption are outside scope, even when they demonstrate implicit physical understanding. The distinguishing criterion is whether model outputs drive autonomous action: a model that generates video of a robot completing a task is out of scope; a model whose predictions govern the actual robot’s behavior is within scope.

Language model systems without persistent world state or physical action coupling are covered by existing frameworks and are outside the scope of this taxonomy. Agentic systems that coordinate

sequences of language model calls without an underlying world model are addressed by the OWASP Agentic AI Top 10 and are similarly outside scope, though hybrid systems that combine agentic orchestration with world model components fall partially within scope for the world model components specifically.

5.3 Limitations

This taxonomy represents a first version of a framework for an emerging class of system. Several limitations should be acknowledged.

The severity ratings assigned to each WM entry reflect qualitative assessment based on the potential consequences of successful exploitation and the current difficulty of detection and mitigation. They are not derived from empirical incident data, which does not yet exist at meaningful scale for world model deployments. Practitioners should treat severity ratings as prioritization guidance rather than precise risk measurements, and should calibrate them against the specific operational context of the system under assessment.

The ten attack categories represent the most significant and distinct attack surfaces identifiable at the current state of world model deployment. As the field matures and deployment experience accumulates, additional categories will emerge and existing categories will require refinement. This framework is intended as a living document and will be updated as the operational security picture develops.

Finally, the mitigation strategies provided for each entry describe the current state of defensive practice. Several mitigations, particularly those involving certified adversarial robustness and formal verification of transition model properties, remain active research areas where practical deployment at scale is not yet fully established. Practitioners should evaluate the maturity of each mitigation against the operational requirements of their specific deployment.

6 Conclusion

The security of autonomous AI systems depends on understanding what those systems actually do. World model systems do not process language and return responses. They construct internal simulations of reality, plan sequences of actions within those simulations, and execute those plans in physical environments where failures have physical consequences. The attack surface this creates is distinct from that of language model systems, and it has been largely unaddressed by existing security frameworks.

MYNDRA-LWM introduces ten attack categories organized around the five cognitive components that all world model systems share. Each category identifies a specific failure mode in the world model’s internal architecture, explains how an adversary can induce that failure, and provides concrete guidance for detection and mitigation. The framework is designed to be immediately applicable by security practitioners, without requiring deep familiarity with the underlying machine learning research, while remaining grounded in the architectural properties that make world model systems both capable and vulnerable.

The arrival of world model systems in production environments is not a future concern. Autonomous vehicles, robotic systems, and spacecraft with onboard decision-making capability are deployed today. The security community has a narrow window in which to develop the frameworks, tools, and institutional knowledge needed to protect these systems before incidents at scale make the cost of that preparation obvious. MYNDRA-LWM is a contribution toward closing that window responsibly.

I release this framework under Creative Commons Attribution 4.0. It is free to use, cite, extend, and adapt. Contributions, corrections, and new attack categories identified through operational

experience are welcome through the project repository at github.com/SpyrosArg/myndra-lwm.

How to cite this work

Argyraeos, S. (2026). MYNDRA-LWM: A Practitioner Taxonomy of Security Risks in Large World Model Systems. MYNDRA Framework. <https://spyrosarg.github.io/myndra-lwm/>